Statistical Computing HW2

Name: 莊明儒 Student ID: 111024513

Contents

1 Problem 1 **1**

2 Problem 2 **8**

3 Acknowledgement **16**

4 Appendix **17**

4.1 A 17

1 Problem 1

(a) Kurtosis: $E\left(\frac{X-EX}{\sqrt{var(X)}}\right)^4$**.**

Two estimators can be considered in correspondence to two cases which depend on whether or not $E(X)$ and $var(X)$ are known.

Case 1: $E(X)$ **and** $var(X)$ **are known.**

Let $(X_1, \dots, X_n)$ be a random sample generated by $f(x)$. By weak law of large number, it is trivial to estimate $E\left(\frac{X-EX}{\sqrt{var(X)}}\right)^4$ by

$$\frac{1}{n}\sum_{i=1}^{n}\frac{X_i - E(X)}{\sqrt{var(X)}}.$$

The standard error of this estimator is

$$\sqrt{\frac{1}{n}var\left\{\frac{X_i - E(X)}{\sqrt{var(X)}}\right\}} = \frac{1}{\sqrt{n}}sd\left\{\frac{X_i - E(X)}{\sqrt{var(X)}}\right\},$$

Again by weak law of large number and continuous mapping theorem, we can estimate the standard error of it by

$$\sqrt{\frac{1}{n}\left[\frac{1}{n-1}\sum_{i=1}^{n}\left\{\left(\frac{X_i - E(X)}{\sqrt{var(X)}}\right)^4 - \frac{1}{n}\sum_{i=1}^{n}\left(\frac{X_i - E(X)}{\sqrt{var(X)}}\right)^4\right\}^2\right]}.$$

Algorithm

step1: Generate $X_1, \dots, X_n$ from $f(x)$.

step2: Calculate $\frac{1}{n}\sum_{i=1}^{n}\left(\frac{X_i - E(X)}{\sqrt{var(X)}}\right)^4$ for the estimate of $E\left(\frac{X-EX}{\sqrt{var(X)}}\right)^4$.

step3: Calculate $\sqrt{\frac{1}{n}\left[\frac{1}{n-1}\sum_{i=1}^{n}\left\{\left(\frac{X_i-E(X)}{\sqrt{var(X)}}\right)^4 - \frac{1}{n}\sum_{i=1}^{n}\left(\frac{X_i-E(X)}{\sqrt{var(X)}}\right)^4\right\}^2\right]}$ for the estimate of the standard error.

Implementation with $n = 10^6$

```
n = 10^6
set.seed(111024513)
x = rweibull(n, shape = 1.5, scale = 1)
```

```
mean.weibull = function(shape, scale){
  scale*gamma(1+1/shape)
}
var.weibull = function(shape, scale){
  scale^2*(gamma(1+2/shape)-gamma(1+1/shape)^2)
}
```

```
est = mean(((x-mean.weibull(shape = 1.5, scale = 1))/sqrt(var.weibull(shape = 1.5, scale = 1)))^4)
se = sd(((x-mean.weibull(shape = 1.5, scale = 1))/sqrt(var.weibull(shape = 1.5, scale = 1)))^4)/sqrt(n)
```

Finally, it turns out that the estimate is 4.3827482 and the standard error is 0.0309321.

Case 2: $E(X)$ and $var(X)$ are unknown.

Denote by μ_1, μ_2, μ_3 and μ_4 the first, second, third, and fourth moment of X. Note that we can rewrite kurtosis $E\left(\frac{X-EX}{\sqrt{var(X)}}\right)^4$ in terms of raw moments. Hence, we have

$$E\left(\frac{X-EX}{\sqrt{var(X)}}\right)^4 = \frac{\mu_4 - 4\mu_3\mu_1 + 6\mu_2\mu_1^2 - 3\mu_1^4}{\mu_2^2 - 2\mu_2\mu_1^2 + \mu_1^4}$$

Let $\mathcal{X}$, be a $4 \times n$ random array. Each ith row of $\mathcal{X}$ consists of $(X_{i1}, \dots, X_{in})$, a random sample generated by $f(x)$, for $i = 1, \dots, 4$. Note that the rows of $\mathcal{X}$ are mutually independent. By weak law of large number and continuous mapping theorem, it is trivial to estimate $E\left(\frac{X-EX}{\sqrt{var(X)}}\right)^4$ by replacing μ_1, μ_2, μ_3 and μ_4 to the sample moments $\bar{X}_1$, $\bar{X}_2$, $\bar{X}_3$ and $\bar{X}_4$, where

$$\bar{X}_1 = \frac{1}{n}\sum_{j=1}^{n} X_{1j}, \quad \bar{X}_2 = \frac{1}{n}\sum_{j=1}^{n} X_{2j}^2,$$
$$\bar{X}_3 = \frac{1}{n}\sum_{j=1}^{n} X_{3j}^3, \quad \bar{X}_4 = \frac{1}{n}\sum_{j=1}^{n} X_{4j}^4.$$

Then we have the estimator, denoted by $\hat{\tau}$, as follows.

$$\frac{\bar{X}_4 - 4\bar{X}_3\bar{X}_1 + 6\bar{X}_2\bar{X}_1^2 - 3\bar{X}_1^4}{\bar{X}_2^2 - 2\bar{X}_2\bar{X}_1^2 + \bar{X}_1^4}$$

Define $\hat{\tau} = g(\bar{X}_1, \bar{X}_2, \bar{X}_3, \bar{X}_4)$. By delta method, it follows that the variance of this estimator can be derived as follows.

$$var(\hat{\tau}) = \nabla' g(\mu_1, \mu_2, \mu_3, \mu_4)\, \Sigma\, \nabla g(\mu_1, \mu_2, \mu_3, \mu_4)$$

where

$$\begin{aligned}\Sigma = cov(\bar{X}_1, \bar{X}_2, \bar{X}_3, \bar{X}_4) &= cov\left(\frac{1}{n}\sum_{j=1}^{n} X_{1j}, \frac{1}{n}\sum_{j=1}^{n} X_{2j}^2, \frac{1}{n}\sum_{j=1}^{n} X_{3j}^3, \frac{1}{n}\sum_{j=1}^{n} X_{4j}^4\right) \\ &= \frac{1}{n} cov\left(X_{1j}, X_{2j}^2, X_{3j}^3, X_{4j}^4\right)\end{aligned}$$

and

$$\begin{aligned}\nabla g(\mu_1, \mu_2, \mu_3, \mu_4) &= \left(\tfrac{\partial g(x_1,x_2,x_3,x_4)}{\partial x_1} \quad \tfrac{\partial g(x_1,x_2,x_3,x_4)}{\partial x_2} \quad \tfrac{\partial g(x_1,x_2,x_3,x_4)}{\partial x_3} \quad \tfrac{\partial g(x_1,x_2,x_3,x_4)}{\partial x_4}\right)' \Big|_{x_1=\mu_1, x_2=\mu_2, x_3=\mu_3, x_4=\mu_4} \\ &= \left(G_1(\mu) \quad G_2(\mu) \quad G_3(\mu) \quad G_4(\mu)\right)',\end{aligned}$$

where $\mu = (\mu_1 \quad \mu_2 \quad \mu_3 \quad \mu_4)'$ and

$$\begin{aligned}G_1(\mu) &= \frac{(-4\mu_3 + 12\mu_2\mu_1 - 12\mu_1^3)(\mu_2^2 - 2\mu_2\mu_1^2 + \mu_1^4) - (\mu_4 - 4\mu_3\mu_1 + 6\mu_2\mu_1^2 - 3\mu_1^4)(-4\mu_2\mu_1 + 4\mu_1^3)}{(\mu_2^2 - 2\mu_2\mu_1^2 + \mu_1^4)^2} \\ G_2(\mu) &= \frac{6\mu_1^2(\mu_2^2 - 2\mu_2\mu_1^2 + \mu_1^4) - (\mu_4 - 4\mu_3\mu_1 + 6\mu_2\mu_1^2 - 3\mu_1^4)(2\mu_2 - 2\mu_1^2)}{(\mu_2^2 - 2\mu_2\mu_1^2 + \mu_1^4)^2} \\ G_3(\mu) &= \frac{-4\mu_1}{\mu_2^2 - 2\mu_2\mu_1^2 + \mu_1^4} \\ G_4(\mu) &= \frac{1}{\mu_2^2 - 2\mu_2\mu_1^2 + \mu_1^4}.\end{aligned}$$

Then again by weak law of large number, it is trivial to estimate $var(\hat{\tau})$ by replacing $\nabla g(\mu_1, \mu_2, \mu_3, \mu_4)$ to $\nabla g(\bar{X}_1, \bar{X}_2, \bar{X}_3, \bar{X}_4)$ and Σ to

$$S = \frac{1}{n}\frac{1}{n-1}\sum_{j=1}^{n}\begin{pmatrix} \{X_{1j} - \bar{X}_1\}^2 & 0 & 0 & 0 \\ 0 & \{X_{2j} - \bar{X}_2\}^2 & 0 & 0 \\ 0 & 0 & \{X_{3j} - \bar{X}_3\}^2 & 0 \\ 0 & 0 & 0 & \{X_{4j} - \bar{X}_4\}^2 \end{pmatrix}.$$

Note that the off-diagonal of S consists of zeros since $(X_{i1}, \dots, X_{in})$ is independent of $(X_{k1}, \dots, X_{kn})$ for $i \neq k$.

Algorithm

step1: Generate the random array $\mathcal{X}$ from $f(x)$.

step2: Calculate $\frac{\bar{X}_4 - 4\bar{X}_3\bar{X}_1 + 6\bar{X}_2\bar{X}_1^2 - 3\bar{X}_1^4}{\bar{X}_2^2 - 2\bar{X}_2\bar{X}_1^2 + \bar{X}_1^4}$ for the estimate of $E\left(\frac{X - EX}{\sqrt{var(X)}}\right)^4$.

step3: Calculate $\sqrt{\nabla g(\bar{X}_1, \bar{X}_2, \bar{X}_3, \bar{X}_4)'\, S\, \nabla g(\bar{X}_1, \bar{X}_2, \bar{X}_3, \bar{X}_4)}$ for the estimate of the standard error.

Implementation with $n = 10^6$

```
n = 10^6
set.seed(111024513)
x = matrix(rweibull(4*n, shape = 1.5, scale = 1), ncol = n)
```

```
G1 = function(x){
  x1 = x[1]; x2 = x[2]; x3 = x[3]; x4 = x[4]
  return(((-4*x3+12*x2*x1-12*x1^3)*(x2^2-2*x2*x1^2+x1^4)-(x4-4*x3*x1+6*x2*x1^2-3*x1^4)
          *(-4*x2*x1+4*x1^3))/((x2^2-2*x2*x1^2+x1^4)^2))
}
G2 = function(x){
  x1 = x[1]; x2 = x[2]; x3 = x[3]; x4 = x[4]
  return((6*x1^2*(x2^2-2*x2*x1^2+x1^4)-(x4-4*x3*x1+6*x2*x1^2-3*x1^4)*(2*x2-2*x1^2))
         /((x2^2-2*x2*x1^2+x1^4)^2))
}
G3 = function(x){
  x1 = x[1]; x2 = x[2]; x3 = x[3]; x4 = x[4]
  return(-4*x1/(x2^2-2*x2*x1^2+x1^4))
}
G4 = function(x){
  x1 = x[1]; x2 = x[2]; x3 = x[3]; x4 = x[4]
  return(1/(x2^2-2*x2*x1^2+x1^4))
}
```

```
x.bar = c(mean(x[1,]), mean(x[2,]^2), mean(x[3,]^3), mean(x[4,]^4))
S = diag(c(var(x[1,]), var(x[2,]^2), var(x[3,]^3), var(x[4,]^4)))/n
grad = matrix(c(G1(x.bar), G2(x.bar), G3(x.bar), G4(x.bar)), ncol = 1)
```

```
num = (x.bar[4]-4*x.bar[3]*x.bar[1]+6*x.bar[2]*x.bar[1]^2-3*x.bar[1]^4)
den = (x.bar[2]^2-2*x.bar[2]*x.bar[1]^2+x.bar[1]^4)
est = num/den
se = sqrt(t(grad) %*% S %*% grad)
```

Finally, it turns out that the estimate is 4.3924668 and the standard error is 0.1522638. (There is an alternative approach for this case provided in Appendix A, which TA may skip.)

(a1) EX^3**.**

Let $(X_1, \dots, X_n)$ be a random sample generated by $f(x)$. By weak law of large number, it is trivial to estimate EX^3 by

$$\frac{1}{n}\sum_{i=1}^{n} X_i^3.$$

The standard error of this estimator is

$$\sqrt{\frac{1}{n} var\left\{X_i^3\right\}} = \frac{1}{\sqrt{n}} sd\left\{X_i^3\right\},$$

Again by weak law of large number and continuous mapping theorem, we can estimate the standard error of it by

$$\sqrt{\frac{1}{n}\left[\frac{1}{n-1}\sum_{i=1}^{n}\left\{X_i^3 - \frac{1}{n}\sum_{j=1}^{n} X_j^3\right\}^2\right]}.$$

Algorithm

step1: Generate $X_1, \dots, X_n$ from $f(x)$.

step2: Calculate $\frac{1}{n}\sum_{i=1}^{n} X_i^3$.

step3: Calculate $\sqrt{\frac{1}{n}\left[\frac{1}{n-1}\sum_{i=1}^{n}\left\{X_i^3 - \frac{1}{n}\sum_{j=1}^{n} X_j^3\right\}^2\right]}$ for the estimate of the standard error.

Implementation with $n = 10^6$

```
n = 10^6
set.seed(111024513)
x = rweibull(n, shape = 1.5, scale = 1)
```

```
est = mean(x^3)
se = sd(x^3)/sqrt(n)
```

Finally, it turns out that the estimate is 1.993856 and the standard error is 0.0044737.

(b) Fisher information matrix (2 × 2):

Let $(X_1, \dots, X_n)$ be a random sample generated by $f(x)$. By weak law of large number and according to the supplementary materials, it is trivial to estimate the Fisher information matrix by

$$\frac{1}{n}\sum_{i=1}^{n} - \begin{pmatrix} \frac{\beta}{\alpha^2}\left[1-(\beta+1)\left(\frac{X_i}{\alpha}\right)^{\beta}\right] & -\frac{1}{\alpha}\left[1-\left(\frac{X_i}{\alpha}\right)^{\beta}-\beta\left(\frac{X_i}{\alpha}\right)^{\beta}\log\left(\frac{X_i}{\alpha}\right)\right] \\ -\frac{1}{\alpha}\left[1-\left(\frac{X_i}{\alpha}\right)^{\beta}-\beta\left(\frac{X_i}{\alpha}\right)^{\beta}\log\left(\frac{X_i}{\alpha}\right)\right] & -\frac{1}{\beta^2}-\left(\log\left(\frac{X_i}{\alpha}\right)\right)^2\left(\frac{X_i}{\alpha}\right)^{\beta} \end{pmatrix}.$$

The element-wise standard error of this matrix estimator is

$$\begin{pmatrix} \sqrt{\frac{1}{n}var\left\{-\frac{\beta}{\alpha^2}\left[1-(\beta+1)\left(\frac{X_i}{\alpha}\right)^{\beta}\right]\right\}} & \sqrt{\frac{1}{n}var\left\{\frac{1}{\alpha}\left[1-\left(\frac{X_i}{\alpha}\right)^{\beta}-\beta\left(\frac{X_i}{\alpha}\right)^{\beta}\log\left(\frac{X_i}{\alpha}\right)\right]\right\}} \\ \sqrt{\frac{1}{n}var\left\{\frac{1}{\alpha}\left[1-\left(\frac{X_i}{\alpha}\right)^{\beta}-\beta\left(\frac{X_i}{\alpha}\right)^{\beta}\log\left(\frac{X_i}{\alpha}\right)\right]\right\}} & \sqrt{\frac{1}{n}var\left\{\frac{1}{\beta^2}-\left(\log\left(\frac{X_i}{\alpha}\right)\right)^2\left(\frac{X_i}{\alpha}\right)^{\beta}\right\}} \end{pmatrix}$$
$$= \begin{pmatrix} \frac{1}{\sqrt{n}}sd\left\{-\frac{\beta}{\alpha^2}\left[1-(\beta+1)\left(\frac{X_i}{\alpha}\right)^{\beta}\right]\right\} & \frac{1}{\sqrt{n}}sd\left\{\frac{1}{\alpha}\left[1-\left(\frac{X_i}{\alpha}\right)^{\beta}-\beta\left(\frac{X_i}{\alpha}\right)^{\beta}\log\left(\frac{X_i}{\alpha}\right)\right]\right\} \\ \frac{1}{\sqrt{n}}sd\left\{\frac{1}{\alpha}\left[1-\left(\frac{X_i}{\alpha}\right)^{\beta}-\beta\left(\frac{X_i}{\alpha}\right)^{\beta}\log\left(\frac{X_i}{\alpha}\right)\right]\right\} & \frac{1}{\sqrt{n}}sd\left\{\frac{1}{\beta^2}-\left(\log\left(\frac{X_i}{\alpha}\right)\right)^2\left(\frac{X_i}{\alpha}\right)^{\beta}\right\} \end{pmatrix}$$

Again by by weak law of large number and continuous mapping theorem, we can estimate the standard error of it by

$$\begin{pmatrix} (1) & (2) \\ (3) & (4) \end{pmatrix}$$

where

$$(1) = \frac{1}{\sqrt{n}}\sqrt{\frac{1}{n-1}\sum_{i=1}^{n}\left\{-\frac{\beta}{\alpha^2}\left[1-(\beta+1)\left(\frac{X_i}{\alpha}\right)^{\beta}\right]-\frac{1}{n}\sum_{j=1}^{n}-\frac{\beta}{\alpha^2}\left[1-(\beta+1)\left(\frac{X_i}{\alpha}\right)^{\beta}\right]\right\}^2}$$

$$(2) = (3) = \frac{1}{\sqrt{n}}\sqrt{\frac{1}{n-1}\sum_{i=1}^{n}\left\{\frac{1}{\alpha}\left[1-\left(\frac{X_i}{\alpha}\right)^{\beta}-\beta\left(\frac{X_i}{\alpha}\right)^{\beta}\log\left(\frac{X_i}{\alpha}\right)\right]-\frac{1}{n}\sum_{j=1}^{n}\frac{1}{\alpha}\left[1-\left(\frac{X_i}{\alpha}\right)^{\beta}-\beta\left(\frac{X_i}{\alpha}\right)^{\beta}\log\left(\frac{X_i}{\alpha}\right)\right]\right\}^2}$$

$$(4) = \frac{1}{\sqrt{n}}\sqrt{\frac{1}{n-1}\sum_{i=1}^{n}\left\{\left[\frac{1}{\beta^2}-\left(\log\left(\frac{X_i}{\alpha}\right)\right)^2\left(\frac{X_i}{\alpha}\right)^{\beta}\right]-\frac{1}{n}\sum_{j=1}^{n}\left[\frac{1}{\beta^2}+\left(\log\left(\frac{X_i}{\alpha}\right)\right)^2\left(\frac{X_i}{\alpha}\right)^{\beta}\right]\right\}^2}.$$

Algorithm

step1: Generate $X_1, \dots, X_n$ from $f(x)$.

step2: Calculate

$$\frac{1}{n}\sum_{i=1}^{n}-\begin{pmatrix} \frac{\beta}{\alpha^2}\left[1-(\beta+1)\left(\frac{X_i}{\alpha}\right)^{\beta}\right] & -\frac{1}{\alpha}\left[1-\left(\frac{X_i}{\alpha}\right)^{\beta}-\beta\left(\frac{X_i}{\alpha}\right)^{\beta}\log\left(\frac{X_i}{\alpha}\right)\right] \\ -\frac{1}{\alpha}\left[1-\left(\frac{X_i}{\alpha}\right)^{\beta}-\beta\left(\frac{X_i}{\alpha}\right)^{\beta}\log\left(\frac{X_i}{\alpha}\right)\right] & -\frac{1}{\beta^2}-\left(\log\left(\frac{X_i}{\alpha}\right)\right)^2\left(\frac{X_i}{\alpha}\right)^{\beta} \end{pmatrix}.$$

for the estimate of $E\left(\frac{X-EX}{\sqrt{var(X)}}\right)^4$.

step3: Calculate (1), (2), (3) and (4) for the estimates of the standard errors.

Implementation with $n = 10^6$

```
n = 10^6
set.seed(111024513)
x = rweibull(n, shape = 1.5, scale = 1)
```

```
fisher.weibull = function(x, shape, scale){
  fisher.arr = array(NA, dim = c(2, 2, n))
  for(i in 1:length(x)){
    fisher.arr[, , i] = -matrix(c((shape/scale^2)*(1-(shape+1)*(x[i]/scale)^shape),
    -(1/scale)*(1-(x[i]/scale)^shape-shape*((x[i]/scale)^shape)*log(x[i]/scale)),
    (-1/scale)*(1-(x[i]/scale)^shape-shape*((x[i]/scale)^shape)*log(x[i]/scale)),
    -(1/(shape^2))-(log(x[i]/scale)^2)*(x[i]/scale)^shape), ncol = 2)
  }
  return(fisher.arr)
}
```

```
est = apply(fisher.weibull(x, shape = 1.5, scale = 1), c(1, 2), mean)
se = apply(fisher.weibull(x, shape = 1.5, scale = 1), c(1, 2), sd)/sqrt(n)
```

Finally, it turns out that the estimate is

$$\begin{pmatrix} 2.2442517 & -0.4189002 \\ -0.4189002 & 0.8092315 \end{pmatrix},$$

and the standard error is

$$\begin{pmatrix} 0.003744221 & 0.0024779419 \\ 0.0024779419 & 0.0009786736 \end{pmatrix}.$$

(c) Evaluate $E[X \mid X > 2]$ using the importance sampling method

Denote by $h(x)$ the pdf of the exponential distribution with shift by 2, i.e.,

$$h(x) = e^{-(x-2)}, \quad x > 2.$$

Let $(X_1, \dots, X_n)$ be a random sample generated by $h(x)$. Note that we have

$$\begin{aligned} E_f[X \mid X > 2] &= \frac{\int_0^\infty xI(x>2)f(x;\theta)dx}{\int_0^\infty I(x>2)f(x;\theta)dx} \\ &= \frac{\int_2^\infty \frac{xf(x;\theta)}{h(x)}h(x)dx}{\int_2^\infty \frac{f(x;\theta)}{h(x)}h(x)dx} \\ &= \frac{E_h\left\{X\frac{f(X;\theta)}{h(X)}\right\}}{E_h\left\{\frac{f(X;\theta)}{h(X)}\right\}}. \end{aligned}$$

Hence, by weak law of large number the numerator and the denominator can be estimated by the corresponding sample means, i.e,

$$\frac{\frac{1}{n}\sum_{i=1}^n X_i \frac{f(X_i;\theta)}{h(X_i)}}{\frac{1}{n}\sum_{i=1}^n \frac{f(X_i;\theta)}{h(X_i)}}.$$

Let $U = \frac{1}{n}\sum_{i=1}^n X_i \frac{f(X_i;\theta)}{h(X_i)}$, $V = \frac{1}{n}\sum_{i=1}^n \frac{f(X_i;\theta)}{h(X_i)}$ and μ, ν the means of U and V respectively. Denote by $\hat{\tau} = \frac{U}{V} = g(U,V)$, the proposed estimator. By delta method, it follows that the variance of this estimator can be derived as follows.

$$var(\hat{\tau}) = \nabla' g(\mu,\nu)\, \Sigma\, \nabla g(\mu,\nu)$$

where

$$\begin{aligned} \Sigma = cov(U,V) &= cov\left(\frac{1}{n}\sum_{i=1}^n X_i \frac{f(X_i;\theta)}{h(X_i)}, \frac{1}{n}\sum_{i=1}^n \frac{f(X_i;\theta)}{h(X_i)}\right) \\ &= \frac{1}{n} cov\left(X_i \frac{f(X_i;\theta)}{h(X_i)}, \frac{f(X_i;\theta)}{h(X_i)}\right) \end{aligned}$$

and

$$\nabla g(\mu,\nu) = \begin{pmatrix} \frac{\partial g(u,v)}{\partial u} & \frac{\partial g(u,v)}{\partial v} \end{pmatrix}' \Big|_{u=\mu, v=\nu} = \begin{pmatrix} \frac{1}{\nu} & -\frac{\mu}{\nu^2} \end{pmatrix}'.$$

Then again by weak law of large number, it is trivial to estimate $var(\hat{\tau})$ by replacing $\nabla g(\mu,\nu)$ to $\left(\frac{1}{V}, -\frac{U}{V^2}\right)'$ and Σ to

$$S = \frac{1}{n}\frac{1}{n-1}\sum_{i=1}^n \begin{pmatrix} (1) & (2) \\ (3) & (4) \end{pmatrix}$$

where

$$
\begin{aligned}
(1) &= \left\{ X_i \frac{f(X_i;\theta)}{h(X_i)} - \frac{1}{n}\sum_{j=1}^{n} X_j \frac{f(X_j;\theta)}{h(X_j)} \right\}^2 \\
(2) = (3) &= \left\{ X_i \frac{f(X_i;\theta)}{h(X_i)} - \frac{1}{n}\sum_{j=1}^{n} X_j \frac{f(X_j;\theta)}{h(X_j)} \right\} \left\{ \frac{f(X_i;\theta)}{h(X_i)} - \frac{1}{n}\sum_{j=1}^{n} \frac{f(X_j;\theta)}{h(X_j)} \right\} \\
(4) &= \left\{ \frac{f(X_i;\theta)}{h(X_i)} - \frac{1}{n}\sum_{j=1}^{n} \frac{f(X_j;\theta)}{h(X_j)} \right\}^2 .
\end{aligned}
$$

Algorithm with $n = 10^6$

step1: Generate $X_1, \dots, X_n$ from $h(x)$.

step2: Calculate $\frac{\frac{1}{n}\sum_{i=1}^{n} X_i \frac{f(X_i)}{h(X_i)}}{\frac{1}{n}\sum_{i=1}^{n} \frac{f(X_i)}{h(X_i)}}$ for the estimate of $E_f[X \mid X > 2]$.

step3: Calculate $\sqrt{\nabla g(U,V)' \, S \, \nabla g(U,V)}$ for the estimate of the standard error.

Implementation

```
f = function(x){
  dweibull(x, shape = 1.5, scale = 1)
}
h = function(x){
  dexp(x-2)
}
```

```
n = 10000
set.seed(111024513)
x = rexp(x)+2
```

```
num.x = x*f(x)/h(x)
den.x = f(x)/h(x)
est = mean(num.x)/mean(den.x)
```

```
grad = matrix(c(1/mean(den.x), -mean(num.x)/(mean(den.x)^2)), ncol = 1)
se = sqrt(t(grad) %*% (cov(cbind(num.x, den.x))/n) %*% grad)
```

Finally, it turns out that the estimate is 2.4312464 and the standard error is 0.0036974.

2 Problem 2

(a) Use the MH algorithm to draw a random sample of size n = 1000 from this bi-variate distribution. Provide the sampling procedure and plot your results.

Denote by $f(x, y)$ the pdf associated with $F(x, y)$. By fundamental theorem of calculus, $f(x, y)$ can be derived as follows.

$$
\begin{aligned}
f(x,y) = \frac{\partial^2 F(x,y)}{\partial x\,\partial y} &= \frac{\partial^2 e^{-V(x,y)}}{\partial x\,\partial y} \\
&= \frac{\partial}{\partial y}\left\{\left(-\frac{\partial V(x,y)}{\partial x}\right)e^{-V(x,y)}\right\} \\
&= \left(\frac{\partial V(x,y)}{\partial x}\cdot\frac{\partial V(x,y)}{\partial y} - \frac{\partial^2 V(x,y)}{\partial x\,\partial y}\right)e^{-V(x,y)}, \quad x>0, \quad y>0,
\end{aligned}
$$

where we have $\frac{\partial V(x,y)}{\partial x}$, $\frac{\partial V(x,y)}{\partial y}$ and $\frac{\partial^2 V(x,y)}{\partial x\,\partial y}$ shown as below with $\Phi(\cdot)$ and $\phi(\cdot)$ respectively the pdf and cdf for standard normal distribution.

$$
\begin{aligned}
\frac{\partial V(x,y)}{\partial x} &= -\frac{1}{x^2}\Phi\left\{\frac{1}{2}-\log\left(\frac{x}{y}\right)\right\} - \frac{1}{x^2}\phi\left\{\frac{1}{2}-\log\left(\frac{x}{y}\right)\right\} + \frac{1}{xy}\phi\left\{\frac{1}{2}-\log\left(\frac{y}{x}\right)\right\} \\
&= -\frac{1}{x^2}\left[\Phi\left\{\frac{1}{2}-\log\left(\frac{x}{y}\right)\right\} + \phi\left\{\frac{1}{2}-\log\left(\frac{x}{y}\right)\right\}\right] + \frac{1}{xy}\phi\left\{\frac{1}{2}-\log\left(\frac{y}{x}\right)\right\},
\end{aligned}
$$

$$
\begin{aligned}
\frac{\partial V(x,y)}{\partial y} &= -\frac{1}{y^2}\Phi\left\{\frac{1}{2}-\log\left(\frac{y}{x}\right)\right\} - \frac{1}{y^2}\phi\left\{\frac{1}{2}-\log\left(\frac{y}{x}\right)\right\} + \frac{1}{xy}\phi\left\{\frac{1}{2}-\log\left(\frac{x}{y}\right)\right\} \\
&= -\frac{1}{y^2}\left[\Phi\left\{\frac{1}{2}-\log\left(\frac{y}{x}\right)\right\} + \phi\left\{\frac{1}{2}-\log\left(\frac{y}{x}\right)\right\}\right] + \frac{1}{xy}\phi\left\{\frac{1}{2}-\log\left(\frac{x}{y}\right)\right\},
\end{aligned}
$$

and

$$
\begin{aligned}
\frac{\partial^2 V(x,y)}{\partial x\,\partial y} &= \frac{\partial}{\partial y}\left(\frac{\partial V(x,y)}{\partial x}\right) \\
&= -\frac{1}{x^2 y}\phi\left\{\frac{1}{2}-\log\left(\frac{x}{y}\right)\right\} - \frac{1}{x^2 y}\phi'\left\{\frac{1}{2}-\log\left(\frac{x}{y}\right)\right\} \\
&\qquad - \frac{1}{xy^2}\phi\left\{\frac{1}{2}-\log\left(\frac{y}{x}\right)\right\} - \frac{1}{xy^2}\phi'\left\{\frac{1}{2}-\log\left(\frac{y}{x}\right)\right\} \\
&= -\frac{1}{x^2 y}\left[\phi\left\{\frac{1}{2}-\log\left(\frac{x}{y}\right)\right\} + \phi'\left\{\frac{1}{2}-\log\left(\frac{x}{y}\right)\right\}\right] \\
&\qquad - \frac{1}{xy^2}\left[\phi\left\{\frac{1}{2}-\log\left(\frac{y}{x}\right)\right\} + \phi'\left\{\frac{1}{2}-\log\left(\frac{y}{x}\right)\right\}\right]
\end{aligned}
$$

with $\phi'(x) = -\frac{x}{\sqrt{2\pi}}e^{-x^2/2}$, the first-order derivative of $\phi(x)$.

```
V = function(x, y){
  return((1/x)*pnorm(1/2-log(x/y))+(1/y)*pnorm(1/2-log(y/x)))
}
partial.F.x = function(x, y){
  return(-(1/x^2)*(pnorm(1/2-log(x/y))+dnorm(1/2-log(x/y)))+(1/(x*y))*dnorm(1/2-log(y/x)))
}
partial.F.y = function(x, y){
  return(-(1/y^2)*(pnorm(1/2-log(y/x))+dnorm(1/2-log(y/x)))+(1/(x*y))*dnorm(1/2-log(x/y)))
}
phi.prime = function(x){
  -(1/sqrt(2*pi))*x*exp((-x^2)/2)
}
partial2.F.x.y = function(x, y){
  return(-(1/((x^2)*y))*(dnorm(1/2-log(x/y))+phi.prime(1/2-log(x/y)))
  -(1/(x*(y^2)))*(dnorm(1/2-log(y/x))+phi.prime(1/2-log(y/x))))
```

```
}
f = function(x, y){
  return((-partial2.F.x.y(x, y)+partial.F.x(x, y)*partial.F.y(x, y))*exp(-V(x, y)))
}
```

Figure 1 shows the look of $f(x, y)$.

```
x = seq(10^-8, 5, length.out = 100)
y = seq(10^-8, 5, length.out = 100)
z = outer(x, y, f)
par(mfrow = c(1, 2))
image(x, y, z)
contour(x, y, z, levels= round(quantile(z, 0.05*(1:20)),2),
        xlab="x", ylab="y", cex.lab=1.5, add = T)
persp(x, y, z)
```

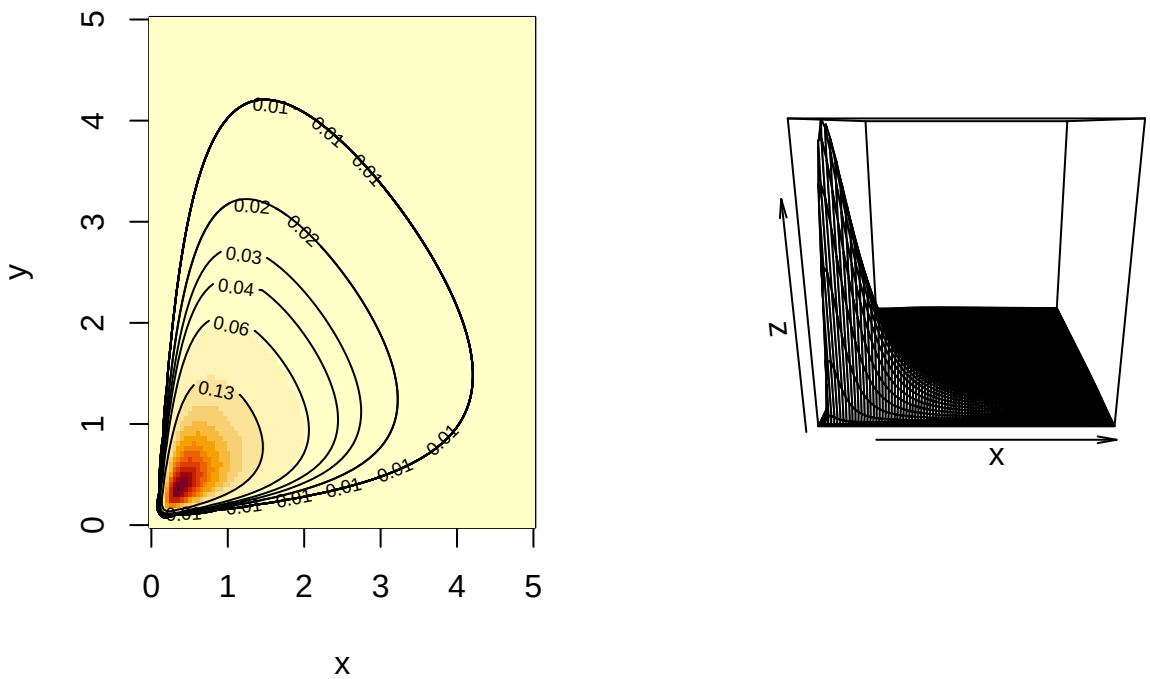
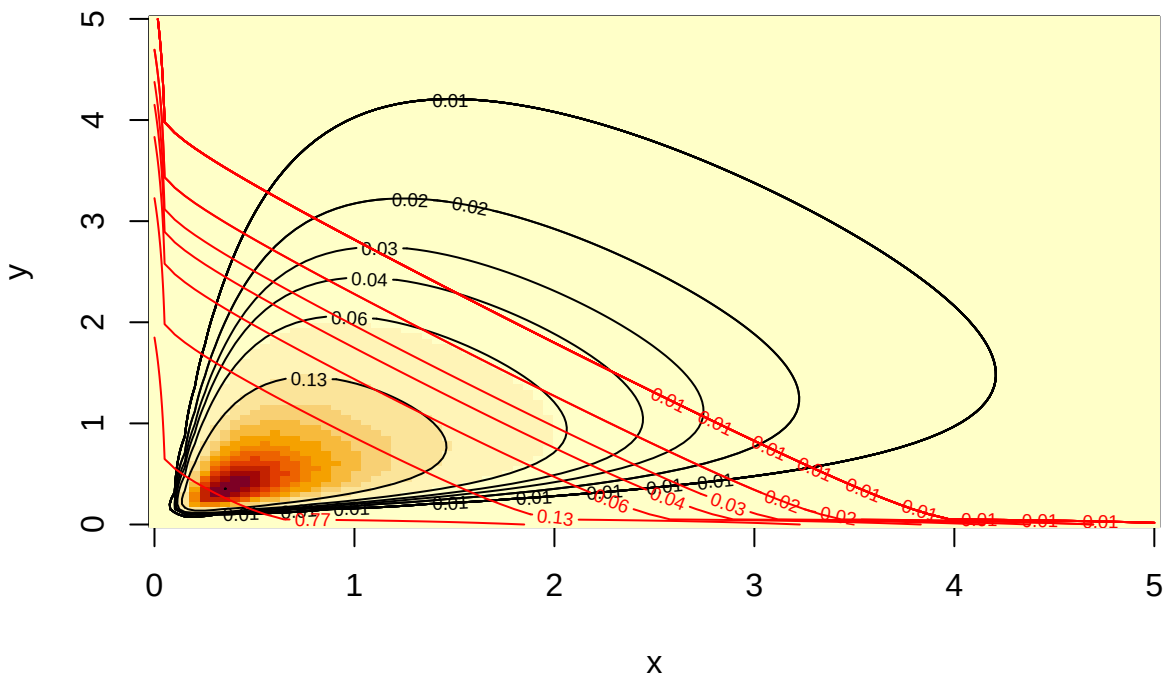


Figure 1: Visualization of $f(x, y)$

Since the support of $f(x, y)$ is $(0, \infty) \times (0, \infty)$, we can select a pdf $q(x, y)$ from the family of gamma distributions. After a few trials, it comes up to us that $\Gamma(\alpha = 0.9, \beta = 0.8)$ seems to be a satisfactory one. Figure 2 shows the contour plots of $f(x, y)$ and $q(x, y)$ together.

```r
q = function(x, y){
  dgamma(x, shape = 0.9, scale = 0.8)*dgamma(y, shape = 0.9, scale = 0.8)
}
```

```r
image(x, y, z)
contour(x, y, z, levels= round(quantile(z, 0.05*(1:20)),2),
        xlab="x", ylab="y", cex.lab=1.5, add = T)
contour(x, y, outer(x, y, q), levels= round(quantile(z, 0.05*(1:20)),2),
        xlab="x", ylab="y", cex.lab=1.5, add = T, col = "red")
```


0.01
0.02
0.02
0.03
0.04
0.06
0.13
0.01
0.01
0.01

Procedure regarding evaluation of whether or not the proposed MH algorithm has converged to the target distribution

step1: Repeat the proposed MH algorithm for 8 times, and thus generate 8 Markov chains.

step2: Use the 8 Markov chains to calculate the change of Gelman-Rubin statistic along with the length of the chains.

step3: Inspect the length at which the Gelman-Rubin statistic seems convergent to 1. Keep only the parts of the 8 Markov chains after that length.

After step3, the resulting sample is believed to be equivalent to that would be generated directly from the target distribution.

Procedure regarding evaluation of the correlation within the sample

step1: Plot the acf plots for the sample.

step2: Take the sample points by a suitable interval such that the sample taken has no significant autocorrelation of lag greater than or equal to 1 and the sample size is as required (1000).

Implementation

First generate the 8 Markov chains as described with $n = 10^6$.

```
mh = function(n){
  init = c(1, 1)
  x.m = matrix(NA, nrow = n, ncol = 2)
  x.m[1,] = init
  for(i in 2:n){
    u = rgamma(2, shape = 0.9, scale = 0.8)
    r = (f(u[1], u[2])/f(x.m[i-1, 1], x.m[i-1, 2]))*(q(x.m[i-1, 1], x.m[i-1, 2])/q(u[1], u[2]))
    if(runif(1)<min(1,r)){
      x.m[i,] = u
    }else{
      x.m[i,] = x.m[i-1,]
    }
  }
  return(x.m)
}
```

```
n = 10^6
x.m1 = mh(n)
x.m2 = mh(n)
x.m3 = mh(n)
x.m4 = mh(n)
x.m5 = mh(n)
x.m6 = mh(n)
x.m7 = mh(n)
x.m8 = mh(n)
mc12345678 = mcmc.list(mcmc(x.m1), mcmc(x.m2), mcmc(x.m3), mcmc(x.m4),
                       mcmc(x.m5), mcmc(x.m6), mcmc(x.m7), mcmc(x.m8))
dump(c("mc12345678"), "problem2_MC2_0.9_0.8.Rdata")
```

The change of Gelman-Rubin stastic along with the length of the Markov chains is shown in Figure 3. It seems that the Markov chains are convergent to the target distribution after the length is greater 7×10^5. Hence, keep only the last 3×10^5 sample points for the 8 Markov chains, resulting in 8 shortened Markov chains.

```
gelman.plot(mc12345678)
```

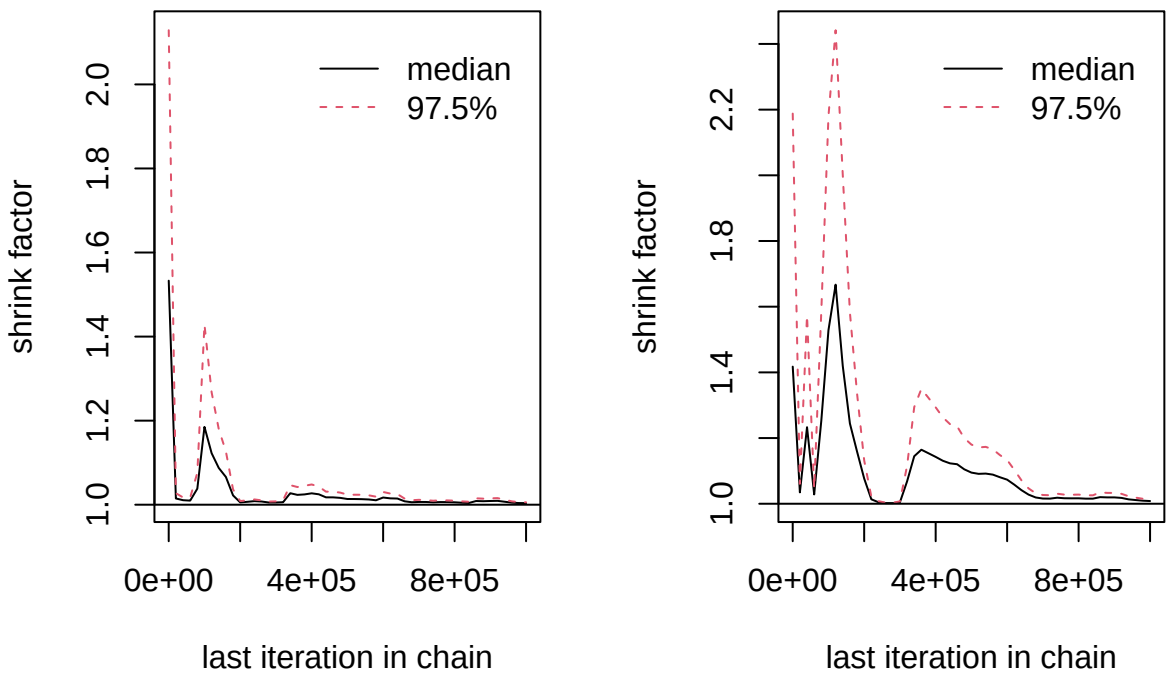


Figure 3: The change of Gelman-Rubin stastic along with the length of the Markov chains

After a few trials and errors, it turns out that by picking the sample points from each of the shortened Markov chains by every 125 ones, we can come up with an nearly i.i.d sample with size 1000, the final sample as required. Figure 4 shows the acf plots for the final sample. We can see that the autocorrelations of lag greater than or equal to 1 are not significant.

```
x.m = rbind(tail(mc12345678[[1]], 3*10^5)[seq(1, 3*10^5, length.out = 125),],
            tail(mc12345678[[2]], 3*10^5)[seq(1, 3*10^5, length.out = 125),],
            tail(mc12345678[[3]], 3*10^5)[seq(1, 3*10^5, length.out = 125),],
            tail(mc12345678[[4]], 3*10^5)[seq(1, 3*10^5, length.out = 125),],
            tail(mc12345678[[5]], 3*10^5)[seq(1, 3*10^5, length.out = 125),],
            tail(mc12345678[[6]], 3*10^5)[seq(1, 3*10^5, length.out = 125),],
            tail(mc12345678[[7]], 3*10^5)[seq(1, 3*10^5, length.out = 125),],
            tail(mc12345678[[8]], 3*10^5)[seq(1, 3*10^5, length.out = 125),])
x.m = x.m[sample(1000),]
acf(x.m)
```

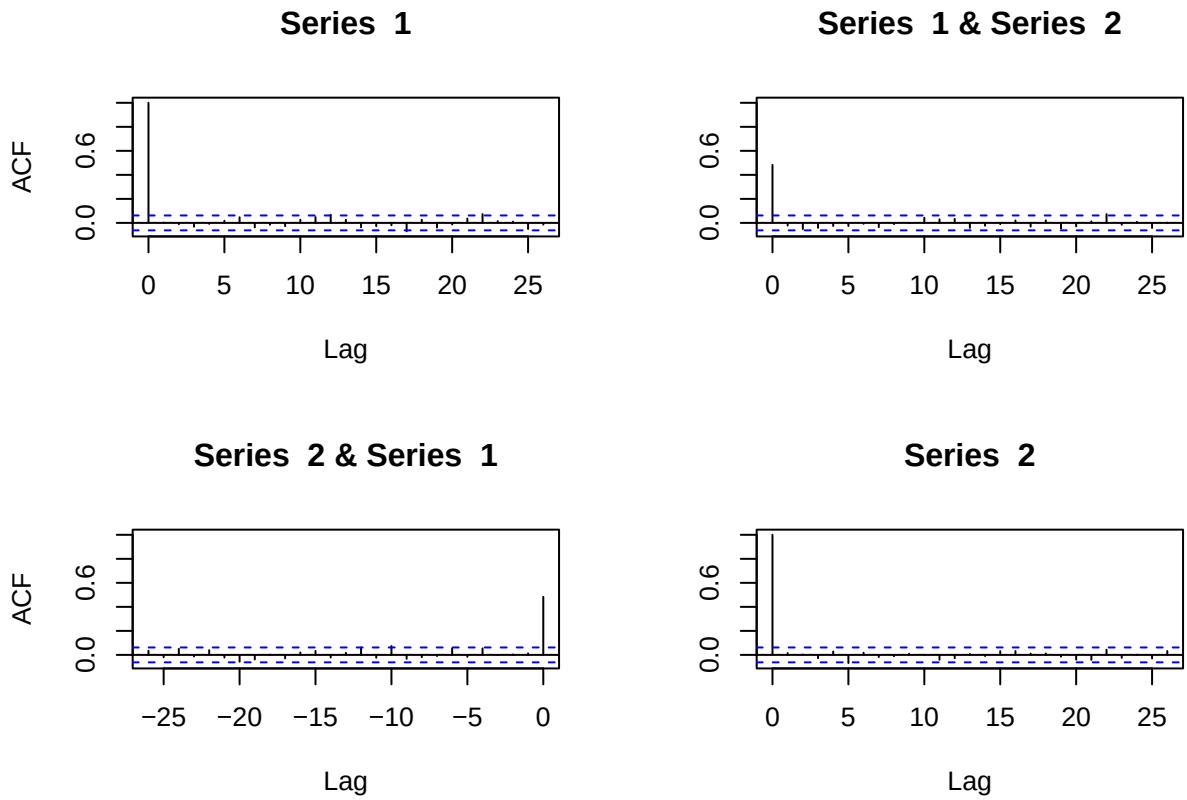


Figure 4: The acf plots for the final sample

Sampling result

Figure 5 shows the final sample by plotting the generated variables on the contour plot of $f(x, y)$. We can see that the distribution of the generated sample looks quite similar to $f(x, y)$.

```
image(x, y, z)
contour(x, y, z, levels= round(quantile(z, 0.05*(1:20)),2),
        xlab="x", ylab="y", cex.lab=1.5, add = T)
points(x.m, pch = 20)
```

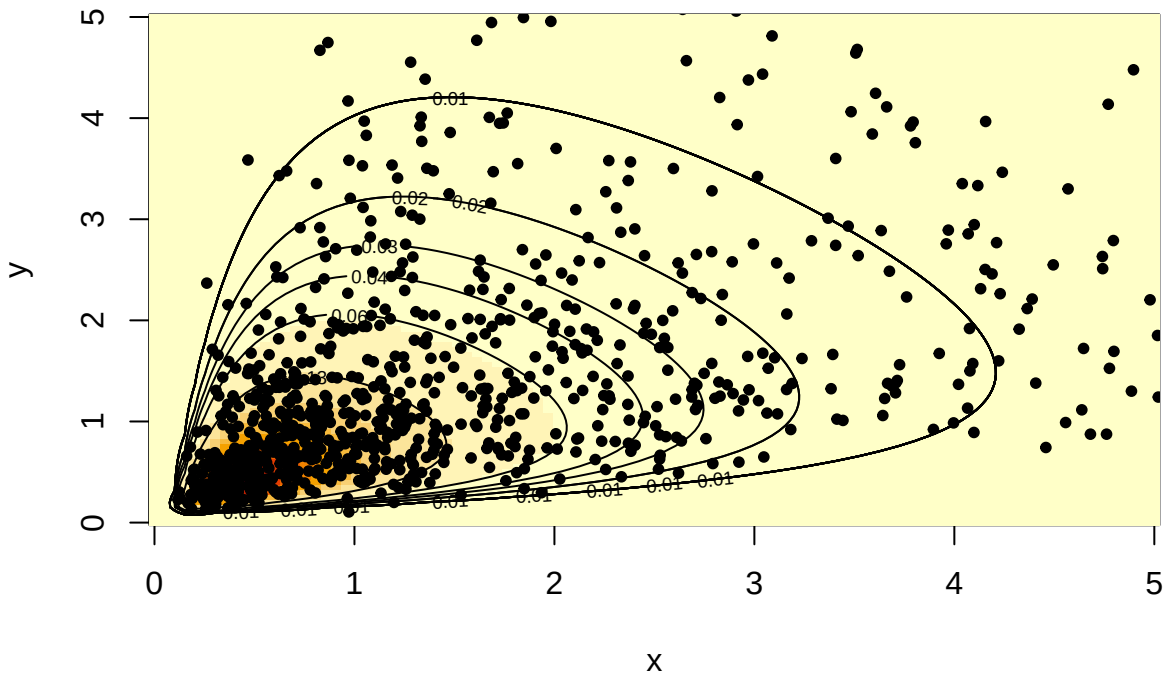


Figure 5: Sampling result

(b) Use the Monte Carlo method to evaluate $\tau \equiv P(X+Y>5)$. Report Monte Carlo estimate $\hat{\tau}$ and its Monte Carlo s.e.

Consider to use importance sampling. It is convenient to select $h(x,y) = q(x,y)$ as the proposal distribution. Note the reason why we consider importance sampling is due to the following result.

$$\begin{aligned} P(X+Y>5) = 1 - P(X+Y \le 5) &= 1 - \int_0^\infty \int_0^\infty I(x+y \le 5) f(x,y) dx dy \\ &= 1 - \int_0^\infty \int_0^\infty I(x+y \le 5) \frac{f(x,y)}{h(x,y)} h(x,y) dx dy \\ &= 1 - E_h \left\{ I(X+Y \le 5) \frac{f(X,Y)}{h(X,Y)} \right\} \end{aligned}$$

Let $\{(X_1,Y_1),\dots,(X_n,Y_n)\}$ be a random sample generated by $h(x,y)$. By weak law of large number, it is trivial to estimate τ by

$$\hat{\tau} = 1 - \frac{1}{n} \sum_{i=1}^n I(X_i + Y_i \le 5) \frac{f(X_i,Y_i)}{h(X_i,Y_i)}$$

The standard error of this estimator is

$$\sqrt{\frac{1}{n} var\left\{I(X_i+Y_i\le 5)\frac{f(X_i,Y_i)}{h(X_i,Y_i)}\right\}} = \frac{1}{\sqrt{n}} sd\left\{I(X_i+Y_i\le 5)\frac{f(X_i,Y_i)}{h(X_i,Y_i)}\right\},$$

Again by weak law of large number and continuous mapping theorem, we can estimate the standard error of $\hat{\tau}$ by

$$\sqrt{\frac{1}{n}\left[\frac{1}{n-1}\sum_{i=1}^{n}\left\{I(X_i+Y_i\le 5)\frac{f(X_i,Y_i)}{h(X_i,Y_i)} - \frac{1}{n}\sum_{j=1}^{n} I(X_j+Y_j\le 5)\frac{f(X_j,Y_j)}{h(X_j,Y_j)}\right\}^2\right]}.$$

Algorithm

step1: Generate $(X_1,Y_1),\dots,(X_n,Y_n)$ from $h(x,y)$.

step2: Calculate $1-\frac{1}{n}\sum_{i=1}^{n} I(X_i+Y_i\le 5)\frac{f(X_i,Y_i)}{h(X_i,Y_i)}$ for the estimate of $P(X+Y>5)$.

step3: Calculate $\sqrt{\frac{1}{n}\left[\frac{1}{n-1}\sum_{i=1}^{n}\left\{I(X_i+Y_i\le 5)\frac{f(X_i,Y_i)}{h(X_i,Y_i)} - \frac{1}{n}\sum_{j=1}^{n} I(X_j+Y_j\le 5)\frac{f(X_j,Y_j)}{h(X_j,Y_j)}\right\}^2\right]}$ for the estimate of the standard error.

Implementation with $n=10^6$

```
f = function(arg){
  x = arg[1]
  y = arg[2]
  return((-partial2.F.x.y(x, y)+partial.F.x(x, y)*partial.F.y(x, y))*exp(-V(x, y)))
}
```

```
h = function(arg){
  x = arg[1]
  y = arg[2]
  return(dgamma(x, shape = 0.9, scale = 0.8)*dgamma(y, shape = 0.9, scale = 0.8))
}
```

```
n= 10^6
set.seed(111024513)
x.m = matrix(rgamma(n*2, shape = 0.9, scale = 0.8), ncol = 2)
```

```
est = 1-mean(as.numeric(apply(x.m, 1, sum) <= 5)*apply(x.m, 1, f)/apply(x.m, 1, h))
se = sd(as.numeric(apply(x.m, 1, sum) <= 5)*apply(x.m, 1, f)/apply(x.m, 1, h))/sqrt(n)
```

Finally, it turns out that the estimate is 0.3432356 and the standard error is 9.4216426×10^{-4}. Moreover, by central limit theorem, the approximated 90% confidence interval can be estimated by

$$[\,\hat{\tau}-1.64\times se(\hat{\tau}),\ \hat{\tau}+1.64\times se(\hat{\tau})\,],$$

which turns out to be [0.3416905, 0.3447808]. Hence, we are 90% confident that the true value of τ lies between 0.3416905 and 0.3447808.

3 Acknowledgement

Thanks for the valuable idea from Mr.Cheng (鄭力綱) for the part (a) in problem 1, where he suggested me to rewrite the kurtosis in terms of moments.

4 Appendix

4.1 A

Case 2: $E(X)$ and $var(X)$ are unknown.

Let $\mathcal{X}$ be a $n \times n$ random array generated by $f(x)$, in which each ith row consists of $(X_i, \dots, X_n)$, for $i = 1, \dots, n$. By weak law of large number, it is trivial to estimate $E\left(\frac{X-EX}{\sqrt{var(X)}}\right)^4$ by

$$\frac{1}{n}\sum_{i=1}^{n}\left(\frac{X_{i1}-\bar{X}_i}{S_i}\right)^4,$$

where $\bar{X}_i = \frac{1}{n}\sum_{j=1}^{n} X_{ij}$ and $S_i = \sqrt{\frac{1}{n-1}\sum_{j=1}^{n}(X_{ij}-\bar{X}_i)^2}$. The standard error of this estimator is

$$\sqrt{\frac{1}{n}var\left\{\left(\frac{X_{i1}-\bar{X}_i}{S_i}\right)^4\right\}} = \frac{1}{\sqrt{n}}sd\left\{\left(\frac{X_{i1}-\bar{X}_i}{S_i}\right)^4\right\}$$

Note that $\left(\frac{X_{i1}-\bar{X}_i}{S_i}\right)^4$ for $i = 1, \dots, n$ are i.i.d. Hence , again by weak law of large number, we can estimate the standard error of it by

$$\frac{1}{\sqrt{n}}\sqrt{\frac{1}{n-1}\sum_{i=1}^{n}\left\{\left(\frac{X_{i1}-\bar{X}_i}{S_i}\right)^4 - \frac{1}{n}\sum_{l=1}^{n}\left(\frac{X_{l1}-\bar{X}_l}{S_l}\right)^4\right\}^2}.$$

Algorithm

step1: Generate the random array $\mathcal{X}$ by f(x).

step2: Calculate $\frac{1}{n}\sum_{i=1}^{n}\left(\frac{X_{i1}-\bar{X}_i}{S_i}\right)^4$ for the estimate of $E\left(\frac{X-EX}{\sqrt{var(X)}}\right)^4$.

step3: Calculate $\frac{1}{\sqrt{n}}\sqrt{\frac{1}{n-1}\sum_{i=1}^{n}\left\{\left(\frac{X_{i1}-\bar{X}_i}{S_i}\right)^4 - \frac{1}{n}\sum_{l=1}^{n}\left(\frac{X_{l1}-\bar{X}_l}{S_l}\right)^4\right\}^2}$ of the estimate of the standard error.

Implementation with $n = 10^4$

```
n = 10^4
set.seed(111024513)
x = matrix(rweibull(n^2, shape = 1.5, scale = 1), ncol = n)
saveRDS(x, "problem1_(a)_x.Rds")
```

```
est = mean(((x[1,]-apply(x, 2, mean))/apply(x, 2, sd))^4)
se = sd(((x[1,]-apply(x, 2, mean))/apply(x, 2, sd))^4)/sqrt(n)
```

Finally, it turns out that the estimate is 4.3311319 and the standard error is 0.0312539.